

Angular Moderno

04 - Criando Componentes

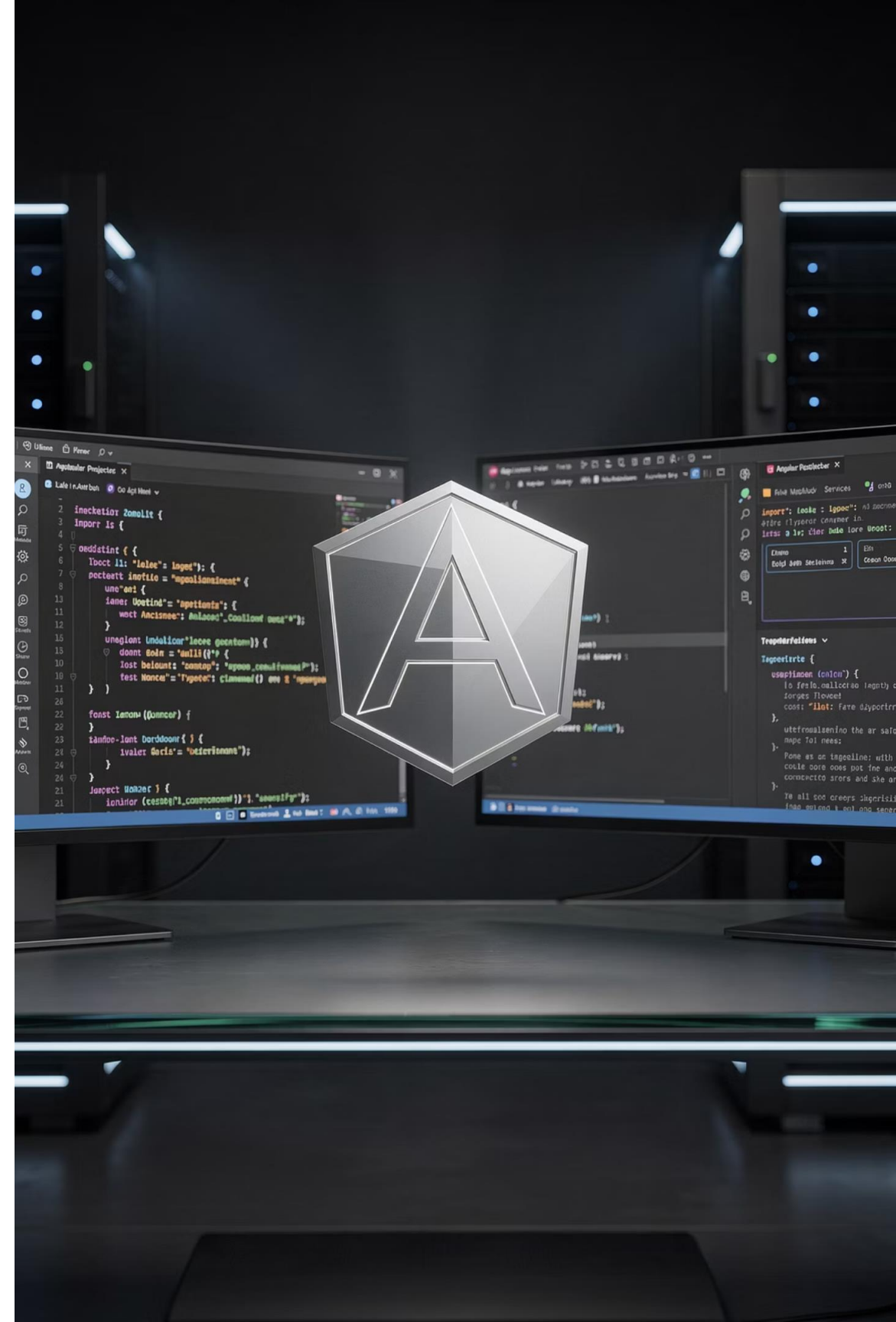


Criando Componentes

Nesta aula, você vai criar seus primeiros componentes Angular utilizando a abordagem **Standalone**, entender como funciona a estrutura de um componente, trabalhar com templates HTML e aplicar **data binding** para exibir dados dinamicamente na tela.

Angular Moderno

04 - Criando Componentes





O que você vai aprender

Nesta aula, você percorrerá o caminho completo de criação e uso de componentes Angular do zero. Ao final, você terá criado seu primeiro componente funcional e exibido dados reais na tela.



Criar Componentes

Gerar e organizar componentes com o Angular CLI



Estrutura do Componente

Entender as três partes: TypeScript, HTML e CSS



Templates HTML

Trabalhar com a interface visual do componente



Data Binding

Conectar dados TypeScript à interface HTML

Antes de Começar

Antes de avançar para a criação dos componentes, confirme que o ambiente de desenvolvimento está funcionando corretamente. Caso algo não esteja certo, resolva antes de continuar para evitar problemas durante a prática.

Projeto da Aula Anterior

Certifique-se de que o projeto criado na aula anterior está presente e funcionando corretamente no seu computador.

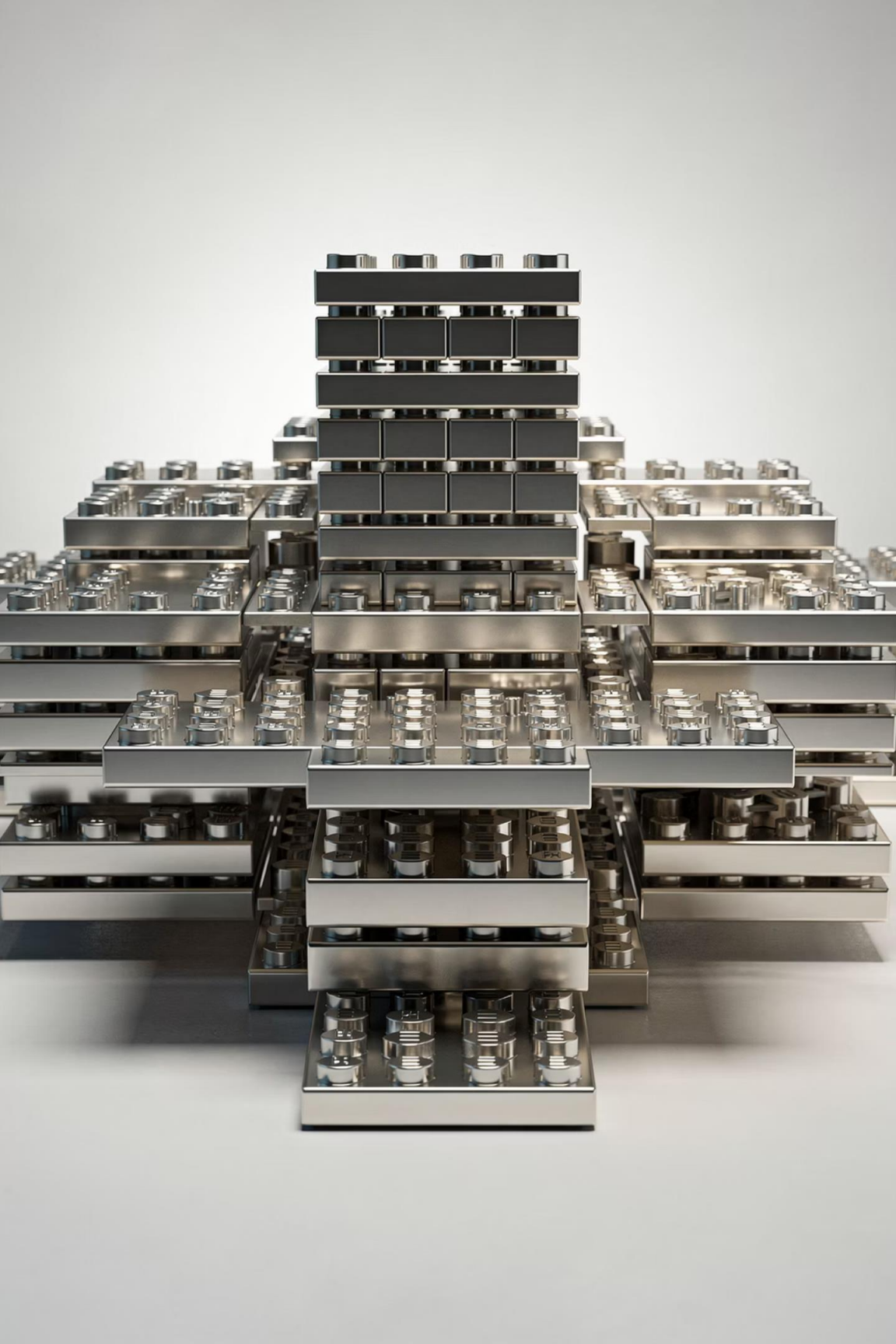
ng serve Rodando

Abra o terminal na pasta do projeto e execute `ng serve`.
Aguarde a mensagem de compilação bem-sucedida.

Navegador Aberto

Acesse `http://localhost:4200` no navegador e confirme que a página inicial do Angular está sendo exibida.

 Se o `ng serve` apresentar erros, verifique se as dependências foram instaladas com `npm install` antes de continuar.



O que é um Componente?

Um **componente** é a unidade fundamental de qualquer aplicação Angular.

Ele representa um bloco visual e funcional independente — como uma peça de Lego que se encaixa com outras para formar a interface completa.

A regra de ouro

Tudo que você vê na tela de uma aplicação Angular é construído com componentes. Botões, listas, menus, cards. Cada elemento pode ser seu próprio componente reutilizável.

Por que isso importa?

Componentes permitem dividir uma interface complexa em partes menores, mais fáceis de entender, manter e reutilizar. Essa organização é a base do desenvolvimento escalável com Angular.

Pensando em Componentes

Uma boa prática é observar qualquer tela de aplicação e identificar suas partes independentes. Cada região com responsabilidade própria pode, e deve, ser um componente separado.



Essa divisão facilita o desenvolvimento em equipe, a reutilização de código e a manutenção futura da aplicação.

O Componente App

Todo projeto Angular começa com um componente raiz chamado `AppComponent`. Ele é automaticamente criado pelo Angular CLI e serve como ponto de entrada de toda a aplicação.

Papel do Componente App

O componente `App` é o componente pai de toda a árvore de componentes. É dentro dele que os demais componentes serão inseridos e compostos para formar a interface final da aplicação.

Arquivos gerados

- `app.ts` — lógica principal
- `app.html` — template raiz
- `app.css` — estilos globais
- `app.spec.ts` — testes

- ❏ Pense no componente `App` como a moldura de um Quadro. Ele contém todos os outros elementos visuais da aplicação.

Criando um Novo Componente

O **Angular CLI** facilita enormemente a criação de componentes. Com um único comando no terminal, todos os arquivos necessários são gerados automaticamente, já configurados e prontos para uso.

1

Abra o Terminal

Certifique-se de estar na pasta raiz do projeto Angular antes de executar qualquer comando.

2

Execute o Comando

Versão completa ou abreviada.
Ambas têm o mesmo resultado.

3

Aguarde a Geração

O CLI cria todos os arquivos automaticamente e confirma no terminal quais foram criados.

```
ng generate component components/produto  
# ou de forma abreviada:  
ng g c components/produto
```

O que esse Comando Faz?

Ao executar o comando `ng g c components/produto`, o Angular CLI gera automaticamente uma pasta organizada com todos os arquivos necessários para o componente funcionar corretamente.



produto.ts

Arquivo TypeScript com a classe do componente. Contém a lógica, os dados e os metadados do componente (decorator `@Component`).



produto.html

Template HTML que define a estrutura visual do componente. É aqui que você escreve o que será exibido na tela.



produto.css

Arquivo de estilos exclusivo do componente. O CSS aqui aplicado não afeta outros componentes da aplicação.



produto.spec.ts

Arquivo TypeScript destinado a testes.

- ✔ Os componentes podem ser criados como standalone.
Em projetos modernos, essa é a abordagem recomendada

Standalone Components na Prática

A partir do Angular 14+ e consolidado no Angular 17+, os componentes **Standalone** são o padrão.

Eles eliminam a necessidade de criar e gerenciar NgModules, tornando a arquitetura mais simples e direta.

Antes (com NgModule)

Era necessário declarar cada componente em um `NgModule`, importar dependências no módulo e gerenciar uma camada extra de configuração.

Agora (Standalone)

O componente é autossuficiente. Ele importa diretamente o que precisa e funciona de forma independente. Menos burocracia, mais produtividade.

Em versões recentes do Angular, os componentes já são criados como standalone por padrão. A propriedade `standalone: true` não aparece explicitamente declarada no decorator (`@Component`), mas o comportamento continua standalone.



Estrutura do Componente

Todo componente Angular é composto por três arquivos principais, cada um com uma responsabilidade clara. Entender essa separação é essencial para desenvolver de forma organizada e profissional.

No nosso exemplo:
produto.ts

Lógica — TypeScript

Define os dados, propriedades e métodos do componente.

É o "cérebro": controla o que acontece e armazena as informações que serão exibidas.

No nosso exemplo:
produto.html

Template — HTML

Define a estrutura visual do componente.

É o "rosto": Determina o que o usuário vê na tela e como os dados serão apresentados.

No nosso exemplo:
produto.css

Estilo — CSS

Define a aparência visual: cores, tamanhos, espaçamentos.

Os estilos são encapsulados e não vazam para outros componentes.

Arquivo TypeScript — A Lógica

O arquivo `.ts` é onde você define os dados e o comportamento do componente. Toda variável declarada aqui pode ser utilizada diretamente no template HTML.

Responsabilidades

- Declarar e inicializar variáveis com os dados
- Criar métodos que respondem a ações do usuário
- Configurar o componente via decorator `@Component`
- Implementar a lógica de negócio da interface

Aqui estamos criando os dados que vão aparecer na tela como exemplo inicial.

Exemplo prático

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-produto',
  imports: [],
  templateUrl: './produto.html',
  styleUrls: ['./produto.css'],
})
export class Produto {
  nome = 'Produto Exemplo';
  preco = 149.99;
}
```

Realize estas alterações em `produto.ts`

Template HTML — A Interface

O arquivo `.html` define o que será renderizado na tela. No Angular, o template vai além do HTML tradicional, pois aceita expressões dinâmicas que exibem dados diretamente do TypeScript.

Template do componente

1 - **APAGUE** os dados originais do template
`produto.html`

2 - Insira esta div que “chama” os dados do produto:

```
<div class="produto">
  <h2>{{ nome }}</h2>
  <p>Preço: R$ {{ preco }}</p>
</div>
```

Aqui estamos elaborando a TELA (html) que exibirá os dados do PRODUTO (que vem de produto.ts)

O que é renderizado

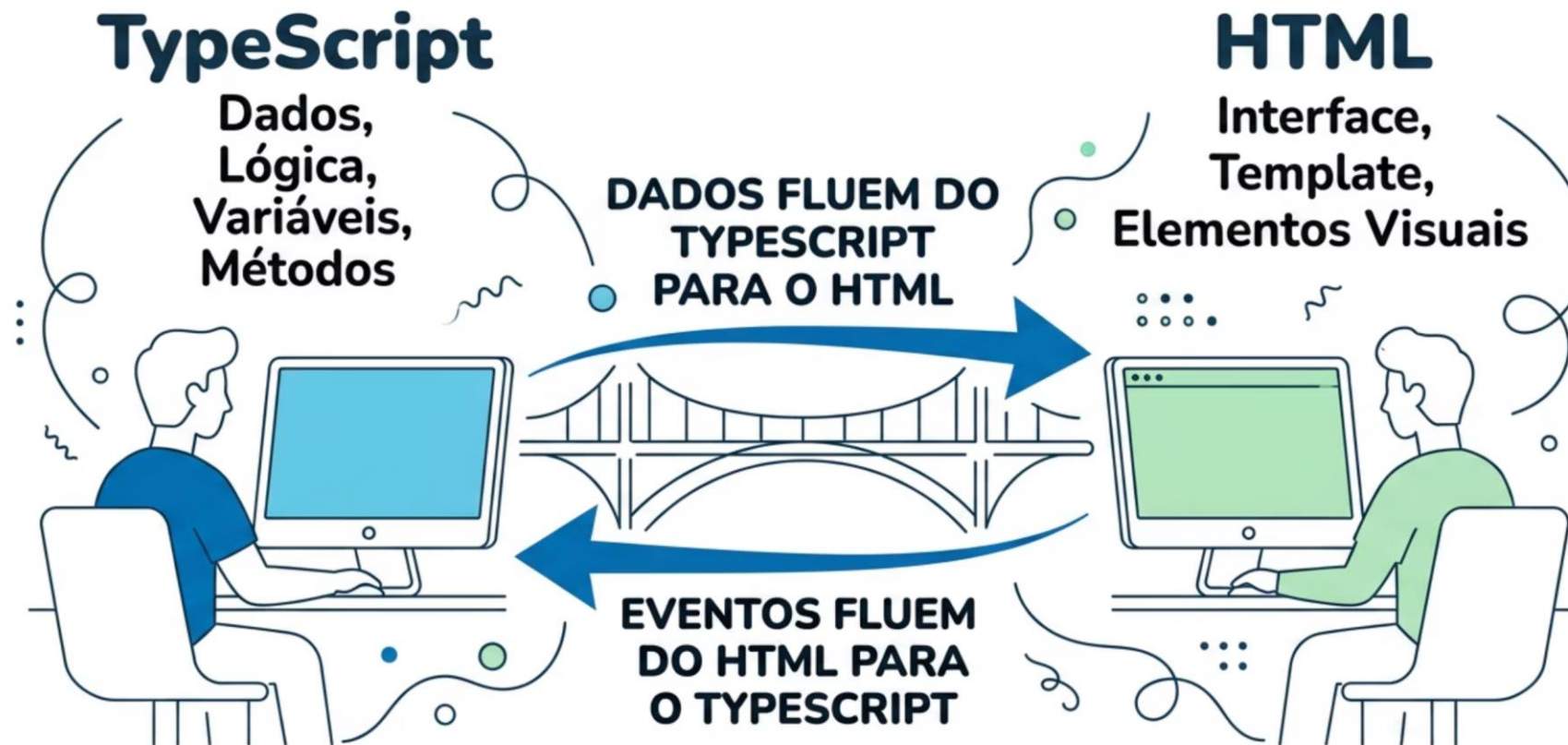
O navegador exibirá o valor real das variáveis no lugar das expressões. Se `nome = "Produto Exemplo"`, o usuário verá:

Produto ExemploPreço: R\$ 149.99

❗ As chaves duplas `{{ }}` são a sintaxe de **interpolação** — elas exibem o valor da variável declarada no TypeScript.

Data Binding — O Conceito

Data Binding é o mecanismo que conecta os dados do TypeScript com a interface HTML. É o que torna o Angular uma ferramenta tão poderosa para criar interfaces dinâmicas e reativas.



Sem Data Binding, você precisaria atualizar manualmente o HTML toda vez que um dado mudasse. O que seria uma tarefa tediosa e propensa a erros. O Angular faz isso automaticamente.

Interpolação — A Forma Mais Simples

A **interpolação** é o tipo mais básico e direto de data binding. Você coloca o nome de uma variável entre duplas chaves no HTML, e o Angular substitui automaticamente pelo seu valor real.

Sintaxe

```
{{ nomeDaVariavel }}
```

O Angular avalia a expressão dentro das chaves e exibe o resultado como texto no HTML.

Expressões válidas

```
{{ nome }}  
{{ preco * 1.1 }}  
{{ 'Olá, ' + usuario }}  
{{ produto.categoria }}
```

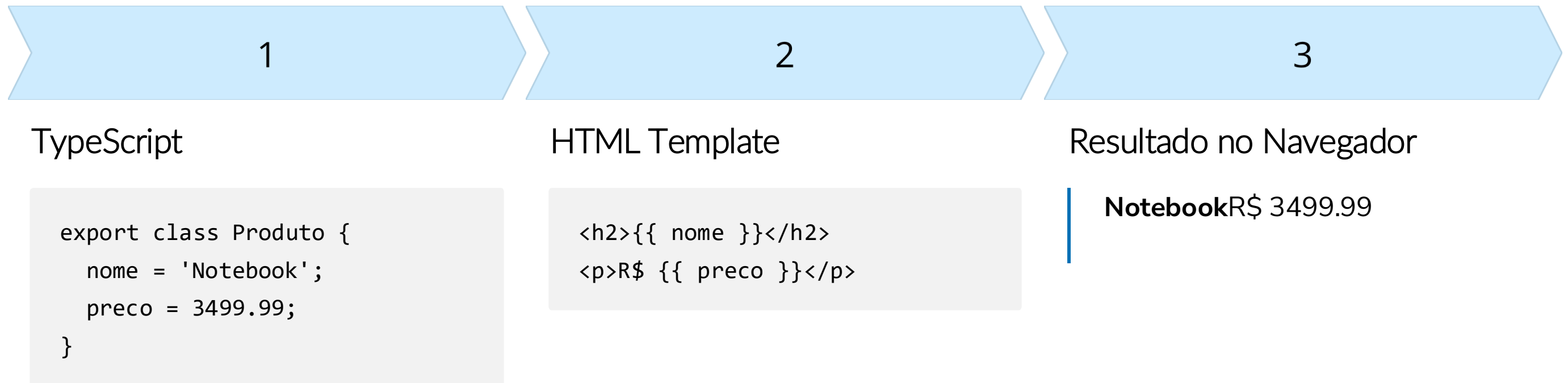
É possível usar operações matemáticas, concatenação e acesso a propriedades de objetos.

O que evitar

Não use atribuições (=), operadores de incremento (++) ou chamadas com efeitos colaterais dentro das interpolações.

Exemplo Completo – Do Dado à Tela

Veja como os três elementos se conectam na prática: o dado é declarado no TypeScript, referenciado no template via interpolação e exibido automaticamente no navegador.



- ✔ Nenhum código extra é necessário. O Angular cuida de toda a atualização automaticamente sempre que o valor da variável mudar.

Inserindo o Componente na Tela (Passo 1 – em app.ts)

Após criar o componente `Produto`, é necessário **utilizá-lo** em algum lugar da aplicação. O mais comum é adicioná-lo na lógica do `App` e no template do `App` para que seja exibido na página principal.

Abra o arquivo `app.ts`

1 - Insira o import junto aos demais imports do início arquivo:

```
import { Produto } from './components/produto/produto';
```

2 – No bloco `@Component`, alterar a declaração de import inserindo `Produto` e removendo a rota:

```
imports: [Produto], // remove RouterOutlet,
```

Inserindo o Componente na Tela (Passo 2 – em app.html)

Agora vamos editar o Template do app para que o produto seja exibido.

1 - Abra o arquivo app.html

Acesse o arquivo `app.html` no seu editor de código. Este é o template do componente raiz que o Angular criou. Vamos apagar todo o seu conteúdo para seguirmos com a nossa edição “do zero”.

APAGUE todo o conteúdo do arquivo `app.html`

2 – Insira a “chamada” ao componente Produto:

```
<h1>Minha Loja</h1>  
<app-produto></app-produto>
```

Resultado Esperado

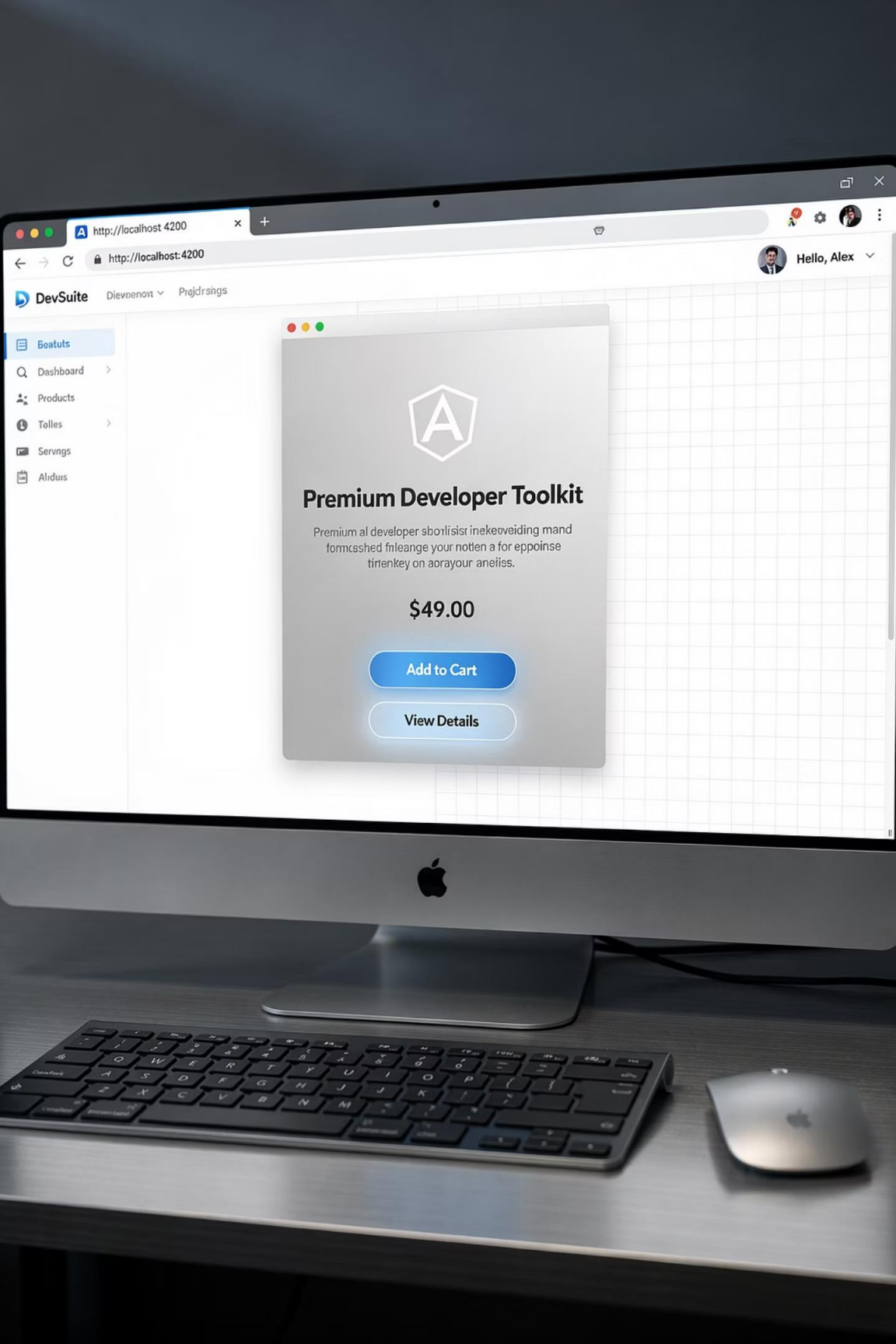
Após salvar o arquivo `app.html` com a tag do seu componente, o Angular recompila automaticamente e o navegador atualiza em tempo real — sem precisar recarregar a página manualmente.

O que você verá

O conteúdo definido no template do componente `Produto` será renderizado na página, exatamente onde você inseriu a tag `<app-produto>`.

Live Reload

Graças ao `ng serve`, toda alteração salva é refletida instantaneamente no navegador sem necessidade de ação manual.



O que foi feito?

Para usar um componente no HTML, ele precisa estar no imports do componente pai.

Essa é a regra que precisamos fixar em nosso raciocínio:

O componente pai (que é toda a estrutura de arquivos da pasta app) é independente do componente que criamos nesta aula.

Querendo que ele seja utilizado na estrutura principal do projeto, as indicações que acabamos de fazer são necessárias:

- 1 – No arquivo .ts do componente (no caso, produto.ts), alterar a classe para inserir os atributos que serão manipulados;
- 2 – No template .html do componente (no caso, produto.html), inserir as marcações para a exibição dos dados do componente;
- 3 – No arquivo .ts app (app.ts), importar o componente;
- 4 – Ainda em app.ts, no bloco @Component, indicar o import do componente.

OBSERVAÇÃO:

Optamos neste momento do aprendizado, por motivos didáticos, em remover as ROTAS que o Angular criou no projeto, pois ainda não trabalhamos este conceito de rotas e isso poderia confundir quem está aprendendo as boas práticas de Angular Moderno.

Futuramente abordaremos todos os conceitos de rotas e faremos as alterações necessárias.

Alterando Dados em Tempo Real

Uma das melhores formas de entender o data binding é testando na prática. Altere o valor de uma variável no TypeScript e observe o que acontece no navegador automaticamente.

Antes da alteração

```
export class Produto {  
  nome = 'Notebook';  
}
```

O navegador exibe: **Notebook**

Após a alteração

```
export class Produto {  
  nome = 'Mouse Gamer';  
}
```

O navegador exibe automaticamente: **Mouse Gamer**

 Salve o arquivo com **Ctrl + S** (ou **Cmd + S** no Mac) e observe a atualização instantânea no navegador em **http://localhost:4200**

O que Acontece Por Baixo dos Panos?

Quando você salva o arquivo, o Angular detecta a mudança, recompila o componente e atualiza apenas as partes da interface que foram afetadas, sem recarregar a página inteira.



Editar TypeScript

Detectar Mudança

Atualizar HTML

Esse processo é chamado de **Change Detection**. É o mecanismo interno do Angular que monitora as variáveis e atualiza a interface sempre que um valor muda, garantindo que a tela esteja sempre sincronizada com os dados.

Por que isso é Tão Importante?

O data binding automático resolve um dos maiores problemas do desenvolvimento web tradicional: manter a interface sincronizada com os dados sem esforço manual.

Sem Angular

Você precisaria buscar o elemento no DOM com `getElementById`, atualizar seu conteúdo manualmente toda vez que o dado mudasse, e garantir que não houvesse inconsistências entre dados e interface.

Com Angular

Você apenas declara a variável e usa `{{ variavel }}` no template. O Angular cuida de todo o resto: monitoramento, atualização e sincronização automática da interface.

- ✔ Isso representa um ganho enorme de produtividade e reduz drasticamente a quantidade de bugs relacionados à interface desatualizada.

Boas Práticas Iniciais

Desenvolver com qualidade desde o início faz uma grande diferença no longo prazo. Adote essas práticas desde o primeiro componente e você terá uma base sólida para projetos maiores.

1

Um Componente por Responsabilidade

Cada componente deve ter uma única função clara. Um componente de produto não deve cuidar do cabeçalho. Isso facilita a manutenção e o reuso.

2

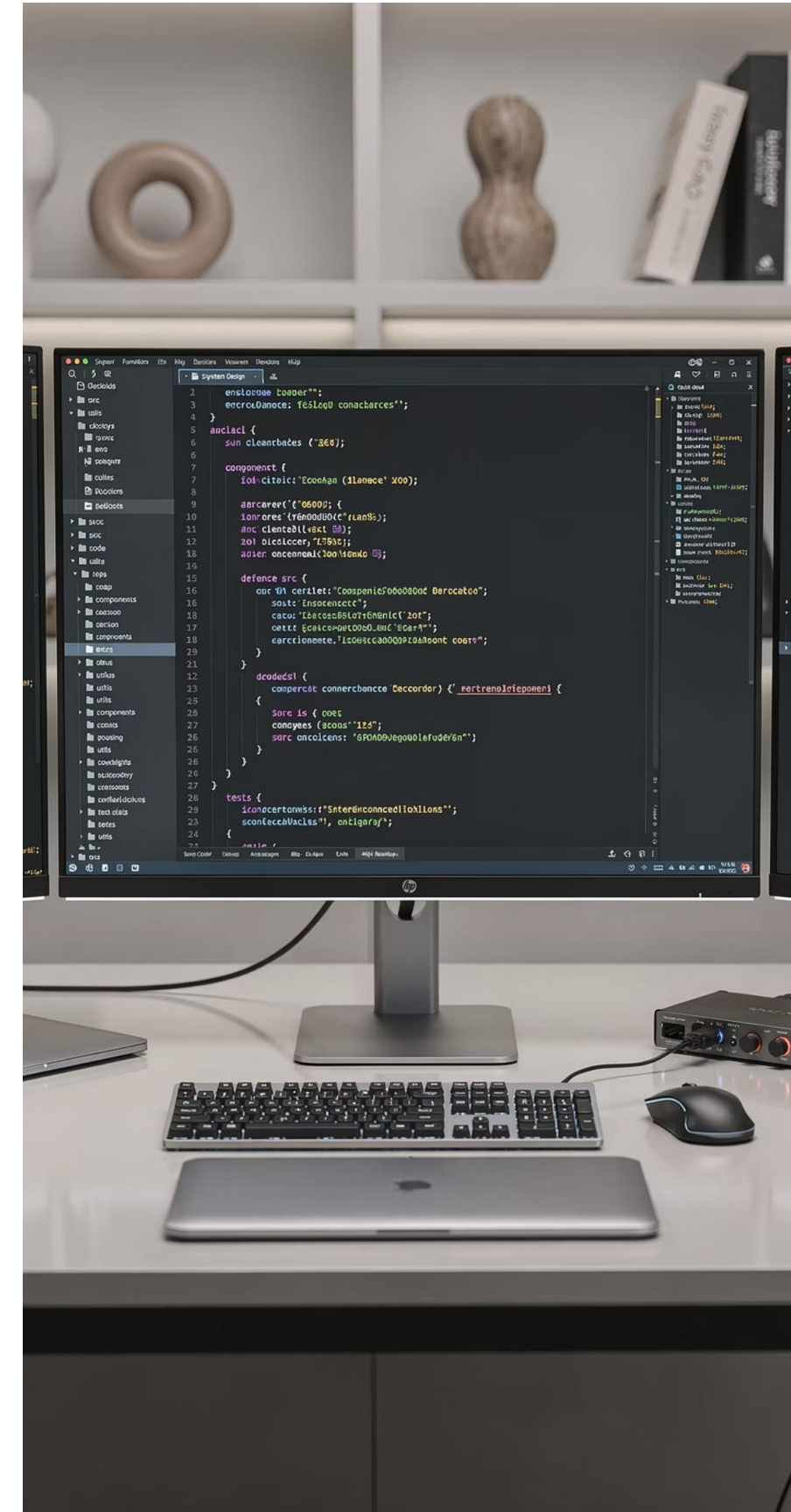
Nomes Claros e Descritivos

Use nomes que descrevam o que o componente faz: `produto`, `carrinho`, `cabecalho`. Evite nomes genéricos como `comp1` ou `teste`.

3

Organização em Pastas

Agrupe componentes relacionados em pastas como `components/`, `pages/` ou `shared/`. Uma estrutura organizada escala bem conforme o projeto cresce.



Recapitulando o que Aprendemos

Você percorreu um caminho completo nesta aula — desde entender o conceito de componentes até criar um funcional e exibir dados na tela. Veja tudo que foi conquistado:

1 Criar Componentes com Angular CLI

Usando `ng g c` para gerar todos os arquivos automaticamente, organizados e configurados como Standalone.

3 Trabalhar com Templates

Escrever HTML dinâmico que exibe dados vindos do TypeScript diretamente na interface do usuário.

2 Entender a Estrutura do Componente

As três camadas: TypeScript (lógica), HTML (template) e CSS (estilos), cada uma com sua responsabilidade bem definida.

4 Usar Data Binding com Interpolação

A sintaxe `{{ variavel }}` conecta os dados da classe ao template de forma automática e reativa.

O que Você Não Precisa se Preocupar Agora

É normal sentir que há muita coisa para aprender. Mas o aprendizado é gradual — e focar no que importa em cada etapa é a chave para evoluir com confiança.

Não se preocupe com:

- Decorar toda a sintaxe do Angular de memória
- Entender profundamente como o Change Detection funciona internamente
- Dominar Angular completo antes de avançar
- Saber todos os tipos de data binding existentes
- Configurações avançadas do `@Component`

Foque em:

- Praticar criando componentes simples
- Entender o fluxo dado → template → tela
- Experimentar e observar os resultados
- Progredir aula a aula com consistência

✓ A fluência vem com a prática. O que parece complexo hoje, será natural em breve.

Parabéns!

Você criou seus primeiros componentes Angular!

Nesta aula você deu um passo fundamental na sua jornada com Angular. Você criou componentes reais, entendeu sua estrutura e fez dados aparecerem na tela de forma dinâmica, usando data binding com interpolação.

Continue praticando

Crie novos componentes, experimente diferentes variáveis e observe o resultado no navegador. A prática é o melhor caminho.



Dúvidas

?

